

My Project — Simple Technical Recap (Phases 0-3)

This is my own cheat sheet. Read this anytime I forget what I built and why.

Phase 0 — Foundation (the skeleton)

What it does: basic app setup — nothing fancy, just the plumbing every app needs.

File	In plain words
<code>core/config.py</code>	One place holding all settings (like passwords, ports) — read from a <code>.env</code> file, never typed directly into code
<code>core/logging_config.py</code>	Makes every log line have a unique "request ID" attached, so I can trace what happened during ONE specific request
<code>core/exceptions.py</code>	My own list of error types (like <code>NotFoundError</code> , <code>AuthenticationError</code>) — instead of every file inventing its own error handling
<code>api/routes/health.py</code>	Two URLs: <code>/health</code> (is the app alive?) and <code>/ready</code> (is it ready for real traffic?)

One sentence to remember: *"Every setting comes from one config file, every error uses one shared error system — nothing is scattered or hardcoded."*

Phase 1 — Simple Auth (login system)

What it does: lets someone log in and proves who they are, without needing a real database yet.

File	In plain words
<code>core/security.py</code>	Turns a password into a scrambled version (hash) using <code>bcrypt</code> — one-way, can't be un-scrambled
<code>core/jwt.py</code>	Makes a "ticket" (JWT token) after login — this ticket proves who you are for 30 minutes, without logging in again each time
<code>infrastructure/repositories/user_repository.py</code>	A simple Python dictionary holding 4 fake test users (one per role) — no real database
<code>services/auth_service.py</code>	The actual login logic: check password, if correct, hand back a ticket
<code>api/middleware/auth_dependencies.py</code>	Checks: is there a valid ticket? Does this role have permission for this action?

The 2 key concepts:

- **Authentication** = "who are you?" (checked once, at login)
- **Authorization / RBAC** = "are you ALLOWED to do this specific thing?" (checked every time, based on your role)

One sentence to remember: *"I hash passwords with bcrypt, issue JWT tickets on login, and check role permissions separately from login — auth and permission-checking are two different jobs."*

Phase 2 — Document Chunking (cutting documents into pieces)

What it does: takes a big document and cuts it into small, searchable pieces.

File	In plain words
<code>domain/document.py</code>	Plain box: "what is one whole document"
<code>domain/chunk.py</code>	Plain box: "what is one small piece," plus tags (which doc, which section)
<code>infrastructure/ingestion/loader.py</code>	Opens <code>.md</code> / <code>.txt</code> files, reads the text out
<code>infrastructure/ingestion/chunker.py</code>	Does the actual cutting — in 2 steps

The 2-step cutting rule (the important part):

1. **First, try cutting by document structure** — look for `##`/`###` headers, keep each section together
2. **If a section is STILL too big** — THEN cut that one section further using a backup method (cut by paragraph, then sentence, then word)

One sentence to remember: *"I use a hybrid chunking strategy — structure-aware first, with recursive character splitting as a fallback for oversized sections — so numbered steps and sections stay together instead of getting randomly cut."*

Phase 3 — Embeddings + Retrieval (searching by meaning)

What it does: turns each chunk into a list of numbers that captures its MEANING, so I can search by meaning instead of just matching exact words.

File	In plain words
<code>domain/ports.py</code>	Two "rulebooks" (interfaces): <code>EmbeddingProvider</code> (rule: "must turn text into numbers") and <code>VectorStore</code> (rule: "must save and search number-lists")
<code>domain/embedding.py</code>	Plain box: one chunk + its number-list, kept together
<code>infrastructure/embedding_providers/bedrock_embedding_provider.py</code>	The REAL code that calls AWS Bedrock's Titan tool to get the numbers
<code>infrastructure/vector_stores/json_vector_store.py</code>	The REAL code that saves/searches the numbers in a simple JSON file (no database yet)
<code>services/retrieval_service.py</code>	The manager: <code>.ingest()</code> (save chunks) and <code>.search()</code> (find matches) — only talks to the rulebooks, never touches AWS or files directly

The key idea — "rulebook vs real worker":

- The **rulebook** (`domain/ports.py`) just says WHAT must happen — it doesn't know about AWS
- The **real worker** (`infrastructure/`) actually DOES it using AWS Bedrock
- Because of this split, I can swap AWS Bedrock for a different tool later, and NONE of my other code needs to change

How search actually works, step by step:

1. Turn the user's question into a number-list (same tool, Bedrock Titan)

2. Compare that number-list against every stored chunk's number-list (this comparison is called "cosine similarity" — just a math formula for "how close are these two lists")
3. Return the closest matches (top 3, by default)

One sentence to remember: *"I built retrieval behind swappable interfaces — the embedding tool (Bedrock) and the storage (JSON file) can each be replaced independently, without touching my service logic, because my service only depends on the interface, not the implementation."*

The ONE pattern that repeats across every phase (this is the big takeaway)

Every phase follows the SAME shape:

```
domain/           → the rulebook (plain, no AWS/database/web code)
services/         → the manager (follows the rulebook, does the real
orchestration)
infrastructure/ → the real workers (actual AWS calls, actual file/database
code)
api/              → the front door (web URLs people actually call)
```

If an interviewer asks "walk me through your architecture" — this diagram IS your answer. Every phase just fills in this same shape with different real workers.

Quick self-test — can I say these in my own words?

- ☐ What's the difference between authentication and authorization?
- ☐ Why does chunking try structure first, then fall back to recursive splitting?
- ☐ Why is `domain/` not allowed to import `boto3` or `json`?
- ☐ What would I need to change to swap AWS Bedrock for a different embedding tool?

If any of these feel shaky, that's fine — just point to this file and we'll re-explain that ONE part, not everything.